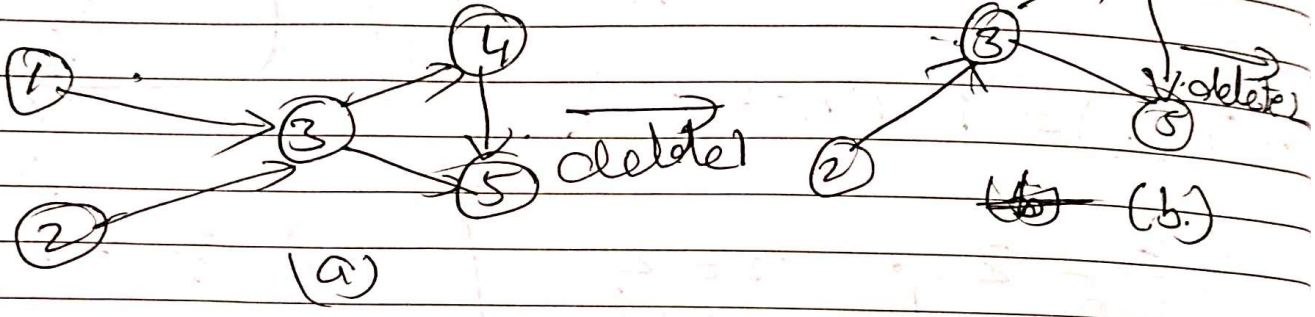
Q. Topological sort using source removal method.

Source removal method: based on decrease-and-conquer technique.

On this method, a vertex with no incoming edges (whose vertex indegree is 0) is selected and deleted along with the outgoing edges. If there are several vertices with no incoming edges, arbitrarily a vertex is selected. The order in which the vertices are visited and deleted one by one result in topological sorting.

Let us take graph.



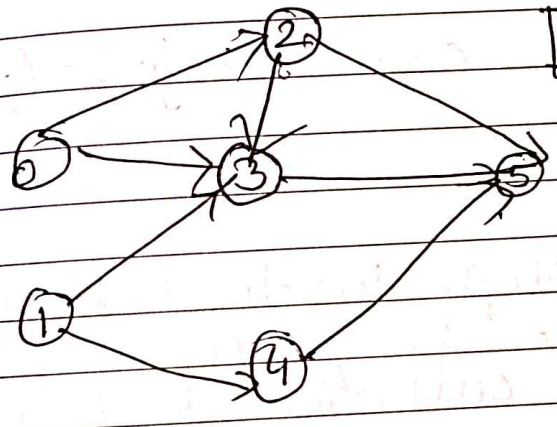
graph showing the pre-tasks to be completed.

It is clear from figure (a) that task 1 & 2 are independent and one of them can be selected and proceed and graph shown in figure (b) is obtained.

Now, the next independent task is 2 (figure b) and can be deleted. The sequence of vertices that are processed and deleted are shown in figure (c) through (f). Thus, the order in which each task has to be completed is given by 1, 2, 3, 4 and 5. The topological sequence is shown below

$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$ (Topological order.

design. Let us consider a graph shown in figure (a), The adjacency matrix for this graph is shown in figure (b)



(a)

Adjacency Matrix

	0	1	2	3	4	5
0	0	0	1	1	0	0
1	0	0	0	1	1	0
2	0	0	0	0	0	1
3	0	0	0	0	0	1
4	0	0	0	0	0	1
5	0	0	0	0	0	0
Sum of Column	0	0	1	3	1	3

indegree of each node

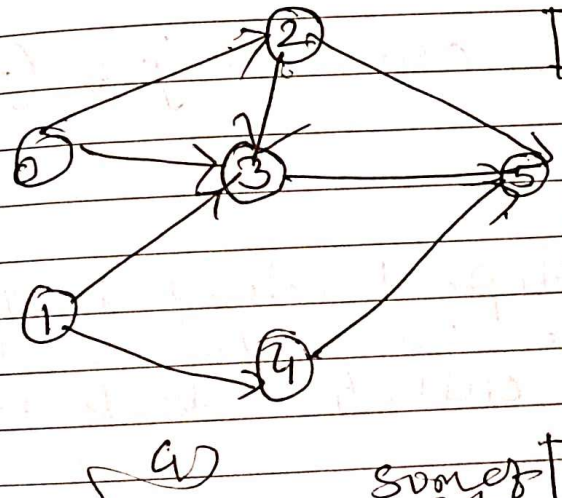
indegree[0] = 0
indegree[1] = 0
indegree[2] = 1
indegree[3] = 3

It is clear from figure (a) that task 1 & 2 are independent and one of them can be selected and proceed and graph shown in figure (b) is obtained.

Now, the next independent task is 2 (figure b) and can be deleted. The sequence of vertices that are processed and deleted are shown in figure (c) through (f). Thus, the order in which each task has to be completed is given by 1, 2, 3, 4 and 5. The topological sequence is shown below

$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$ (Topological order.

design. Let us consider a graph shown in figure (a), The adjacency matrix for this graph is shown in figure (b)



(a)

Adjacency Matrix

	0	1	2	3	4	5
0	0	0	1	1	0	0
1	0	0	0	1	1	0
2	0	0	0	0	0	1
3	0	0	0	0	0	1
4	0	0	0	0	0	1
5	0	0	0	0	0	0
Sum of Column	0	0	1	2	1	3

indgree of each node

indegree[0] = 0
indegree[1] = 0
indegree[2] = 1
indegree[3] = 2

Since indegree[0] = indegree[1] = 0

Since indegree of jobs 0 and 1 is 0 these jobs are considered independent jobs.

indegree[2] = 1 indicates that job 2 depends on one job (namely job 0).

indgree[3] = 3 indicates that job 3 depends on one job (namely 0, 1 & 2)

indgree[4] = 1 indicates that job 4 depends on only one job (i.e 1)

indegree[5] = 3 which indicates that job 5 depends on three jobs (namely 2, 3 & 4).

So, the various steps involved in the algo design have to be performed repeatedly till all the stack is empty.

Process:

Step 1: Find the vertices whose indegree is zero and place them on the stack shown in second column of table.

Step2: Pop a vertex u and it is the task to be done (third column of table)

Step3: Add the vertex u to the solution vector (fourth column of table). This vertex represents the next job to be considered for executing.

Step4: Find the vertices v adjacent to the vertex u. The vertices v represents the jobs which depends on job u (fifth column in the table).

Step5: Decrement indegree[v] by one thereby reducing the number dependencies on v by one. Since, indegree[v] gives the number of jobs to be completed before job v and job u is completed the dependent jobs of v are reduced by one (shown in sixth column in table.

Step6: The detailed activities performed are shown below step by step. In initial steps find the indgree of each node (shown in the last column of table).

Step7: The indegree[2] and indegree[3] are zero indicating jobs 0 and 1 are independent jobs and so are pushed on to the Stack. Then job 1 is deleted from Stack and it is added solution vector. The nodes adjacent to 1 are taken (for example job 3 and 4) Reduce the dependency of v 3 and job 4 by one (see the last.

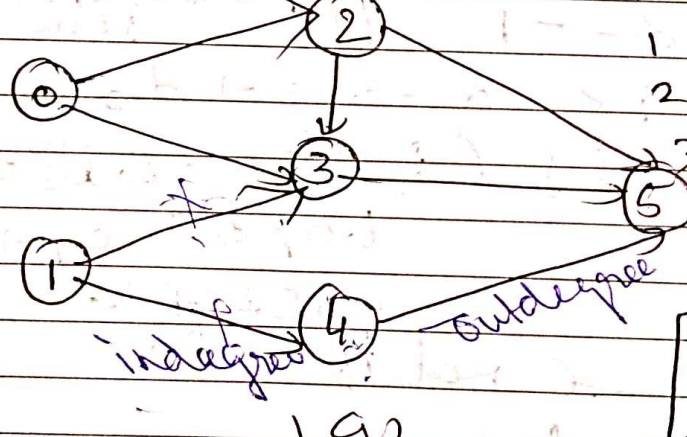
Since indegree[4] is zero. it is pushed on to stack, then delete job 4 from the stack and it is solution. Then, find the vertices adjacent to 4. Here, 5 is adjacent to 4 and so decrement indegree[5], by one and the process is repeated till stack is empty.

Algorithm for topological order.

```
Algorith topological_sort(a, n, s)
// purpose: To obtain the sequence of jobs
to be executed resulting in topological
order.
// input-
// a: adjacency matrix of the given graph
   n: The number of vertices in the graph.
// output: s - indicates the jobs that are
   to be executed in the order.
for j ← 0 to n-1 do (// obtain
    sum ← 0          indegree
                     of each
                     vertex.
for i ← 0 to n-1 do
        sum ← sum + a[i][j]
end for.
indegree[j] ← sum
end for.
top ← -1 // place the independent
                jobs on the stack.
for i ← 0 to n-1 do
    if (indegree[i] = 0)
        top ← top+1
        s[top] ← i)
end if
```

```
while top ≠ 1
    u ← s[top]          // Find the topological sequence.
    top ← top - 1       // Delete the job from the stack
Add u to solution vector T
for each vertex v adjacent to u
    decrement indegree[v] by one.
    if (indegree[v] = 0)
        top ← top + 1
        s[top] ← v
    end if
end for.
end while
print T                 // output the topological sequence.
return
```

Fig-1: graph showing one pre-task to be completed and adjacency relation



Adjacent-matrix

	0	1	2	3	4	5
0	0	0	1	1	0	0
1	0	0	0	1	1	0
2	0	0	0	1	0	1
3	0	0	0	0	0	1
4	0	0	0	0	0	1
5	0	0	0	0	0	0
	0	0	1	3	1	3

Sum of Columns

Steps	Stack	u = POP()	Solution	V = adj(u)	indegree of Jobs [0][1][2][3][4][5]
Initial				Find indegree of each node →	0 0 1 2 1 3
1	0, 1	1	1	3, 4	0 0 1 2 0 3
2	0, 4	4 ✓	1, 4	5	0 0 1 2 0 2
3	0	0	1, 4, 0,	2, 3	0 0 0 1 0 2
4	2	2	1, 4, 0, 2,	3, 5	0 0 0 0 0 1
5	3	3	1, 4, 0, 2, 3,	5	0 0 0 0 0 0
6	5	5	1, 4, 0, 2, 3, 5	—	—

Topological sequence

1 → 1,4 → 1,4,0 → 1,4,0,2 → 1,4,0,2,3 → 1,4,0,2,3,5

10. Apply the DFS based algorithm to solve the topological sorting problem for the following graph.

